



## ASSIGNMENT REPORT

---

# Digital Design

## *Assignment 2 — Up/Down Counter (Verilog Design & Simulation)*

|                 |                                     |
|-----------------|-------------------------------------|
| Course          | Digital Design                      |
| Assignment      | Assignment 2 — Up_Dn_Counter Module |
| Student Name    | Mohamed Hossam                      |
| Instructor / TA | Eng. Ali Temsah                     |
| Submission Date | 10/7/2026                           |

## 01 OBJECTIVE

*This assignment designs, implements, and verifies a 5-bit synchronous up/down counter (Up\_Dn\_Counter) in Verilog, with priority-based Load/Down/Up control and saturating behavior at the bounds 0 and 31, and confirms correct operation through an Icarus Verilog simulation of a dedicated testbench.*

## 02 PROBLEM STATEMENT / REQUIREMENTS

*The task is to design a 5-bit register (Counter[4:0]) whose next value on every rising clock edge is chosen by a fixed priority among three control signals — Load, Down, and Up — with the increment and decrement paths saturating rather than wrapping around at the register's upper and lower bounds.*

- 5-bit up/down counter (Counter[4:0]) with inputs IN[4:0], Load, Up, Down, CLK and outputs Counter[4:0], High, Low.
- Priority on posedge CLK: Load (highest) loads Counter from IN; else Down decrements; else Up increments; else Counter holds.
- Increment saturates at 31 (does not wrap past the maximum); decrement saturates at 0 (does not wrap past the minimum).
- High flag asserts when Counter == 31; Low flag asserts when Counter == 0.
- Verification performed with a self-checking testbench simulated using Icarus Verilog (iverilog/vvp), driven by a free-running clock of period 10 time units.

## 03 DESIGN & ANALYSIS

### 3.1 Design Approach

*The design is a single always block sensitive to posedge CLK, implementing an if–else if–else priority chain rather than a Boolean minimization or separate state machine, since the counter has no internal modes beyond its numeric value. Load is checked first and takes Counter unconditionally from IN. If Load is low, Down is checked next and decrements Counter provided the Low flag is not already set (preventing wraparound below 0). If neither Load nor a valid Down applies, Up is checked and increments Counter provided the High flag is not already set (preventing wraparound above 31). If none of the three conditions hold, Counter retains its value. The High and Low flags are combinational outputs derived directly from the registered Counter value via two equality comparators, so they update immediately after each clock edge without an extra cycle of latency.*

### 3.2 Truth Table / State Table

*This design has no internal states beyond the counter value itself, so a classical state table is not applicable. Instead, the table below summarizes the priority-encoded next-state logic that the always block implements.*

**1. Load = 1** → Counter ← IN (highest priority, checked first, overrides Down/Up)

2. **Load = 0, Down = 1** → Counter ← Counter – 1, unless Low is already set (second priority)
3. **Load = 0, Down = 0, Up = 1** → Counter ← Counter + 1, unless High is already set (lowest priority)
4. **Otherwise** → Counter holds its value; High = (Counter == 31), Low = (Counter == 0), combinational.

### 3.3 Circuit / Block Diagram

The synthesized structure consists of a 5-bit register (D flip-flops) clocked by CLK, fed by a 3-level multiplexer chain selecting between the loaded value (IN), the incremented value (Counter+1, gated by Up and not High), the decremented value (Counter-1, gated by Down and not Low), and the held value (Counter). Load drives the top-priority mux input; the +1/-1 paths are gated by AND gates with the High/Low flags to implement saturation. Two comparators (== 31 and == 0) generate the High and Low flags from the registered Counter output.

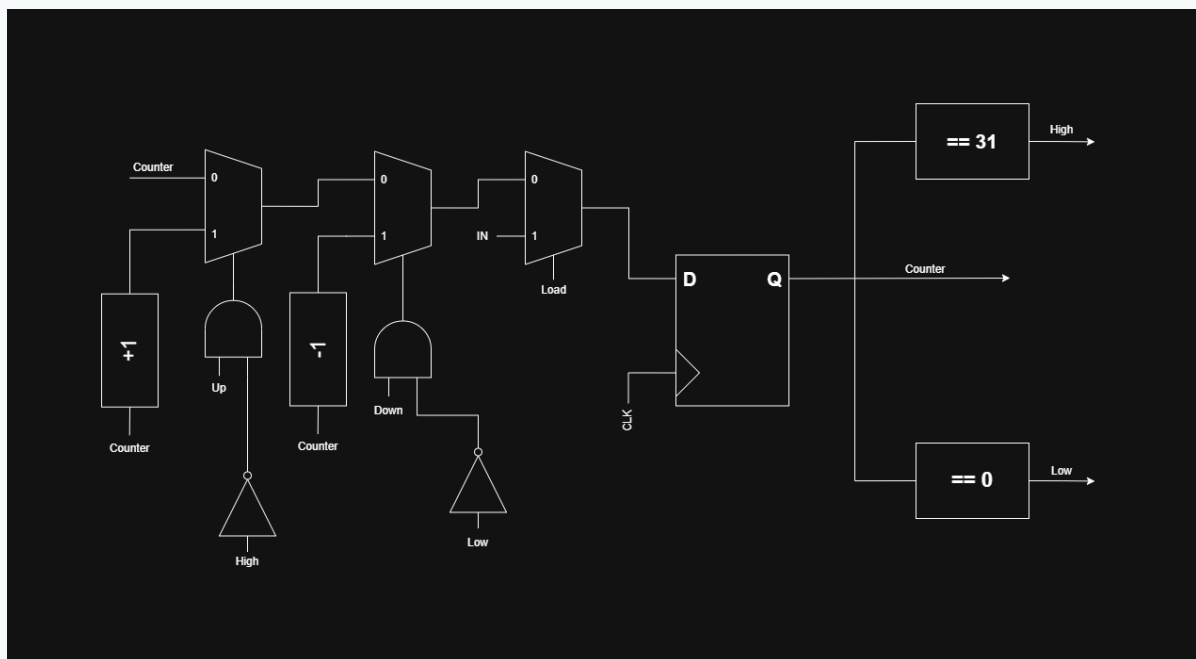


Figure 1: Synthesis (structural) diagram of Up\_Dn\_Counter, drawn in Draw.io

### 3.4 HDL Implementation

The design (Up\_Dn\_Counter.v, below) implements the priority chain directly as nested if-else if statements inside a single always @(posedge CLK) block, with High and Low as continuous (combinational) assign statements. The accompanying testbench (Up\_Dn\_Counter\_TB.v) drives a free-running clock of period 10 and applies a sequence of Load/Down/Up/IN combinations using a fork/join block with delayed assignments, dumps waveforms to a VCD file, and prints a live \$monitor trace of Counter and the High/Low flags.

#### Up\_Dn\_Counter.v — design module

```
module Up_Dn_Counter(
  input [4:0] IN,
```

```

input Load,Up,Down,CLK,
output reg [4:0] Counter,
output High,Low
);
assign High = (Counter == 5'd31);
assign Low = (Counter == 5'd0);
always @(posedge CLK) begin
    if(Load) begin
        Counter <= IN;
    end
    else if (Down && !Low) begin
        Counter <= Counter - 1;
    end
    else if (Up && !High) begin
        Counter <= Counter + 1;
    end
    else Counter <= Counter;
end
endmodule

```

### Up\_Dn\_Counter\_TB.v — testbench

```

module Up_Dn_Counter_TB;
    reg [4:0] IN_TB;
    reg Load_TB,Up_TB,Down_TB,CLK_TB;
    wire [4:0] Counter_TB;
    wire High_TB,Low_TB;
    Up_Dn_Counter DUT_Up_Dn_Counter (
        .IN(IN_TB), .Load(Load_TB), .Up(Up_TB), .Down(Down_TB), .CLK(CLK_TB),
        .Counter(Counter_TB), .High(High_TB), .Low(Low_TB)
    );
    initial begin
        CLK_TB = 0;
        forever begin
            #5 CLK_TB = ~CLK_TB;
        end
    end
    initial begin
        $dumpfile("Up_Dn_Counter_TB.vcd");
        $dumpvars;
    end
    initial begin
        $monitor("At Time: %0t,Counter: %0b,High Flag: %0b,Low Flag: %0b",
            $time,Counter_TB,High_TB,Load_TB);
    end
    initial fork
        #0 {Load_TB,IN_TB} = 6'b110101;
        #10 {Load_TB,Down_TB,Up_TB} = 3'b111;
        #20 {Load_TB,Down_TB,Up_TB} = 3'b010;
        #30 {Load_TB,Down_TB,Up_TB} = 3'b011;
        #40 {Load_TB,Down_TB,Up_TB} = 3'b001;
        #50 {Load_TB,Down_TB,Up_TB} = 3'b000;
        #60 {Load_TB,IN_TB} = 6'b100001;
        #70 {Load_TB,Down_TB,Up_TB} = 3'b010;
        #110 {Load_TB,IN_TB} = 6'b111110;
        #130 {Load_TB,Down_TB,Up_TB} = 3'b001;
        #200 $finish;
    join
endmodule

```

## 04 SIMULATION & RESULTS

Verification was carried out with Icarus Verilog (iverilog/vvp) using the free-running-clock testbench described above. The stimulus sequence exercises every priority case: Load asserted alone, Load asserted together with Down/Up (confirming Load wins), Down and Up asserted together (confirming Down wins), Up alone, a hold cycle with no control signals asserted, and two additional Load/Down and Load/Up passes that drive the counter toward its 0 and 31 saturation boundaries.

| Time (ns) | Load / Down / Up     | IN    | Purpose                                                  | Simulated Result                                         |
|-----------|----------------------|-------|----------------------------------------------------------|----------------------------------------------------------|
| 0         | Load=1               | 10101 | Load Counter with 21 (10101)                             | Counter = 10101 (21) at t=5                              |
| 10        | Load=1, Down=1, Up=1 | —     | Load still asserted (highest priority) — Down/Up ignored | Counter unchanged at 10101 (21); Load priority confirmed |
| 20        | Down=1               | —     | Down only: decrement (Down > Up priority)                | Counter = 10100 (20) at t=25                             |
| 30        | Down=1, Up=1         | —     | Down asserted with Up: Down wins, decrement              | Counter = 10011 (19) at t=35                             |
| 40        | Up=1                 | —     | Up only: increment                                       | Counter = 10100 (20) at t=45                             |
| 50        | (none)               | —     | Hold: Counter unchanged                                  | Counter remains 10100 (20)                               |
| 60        | Load=1               | 00001 | Load Counter with 1 (00001)                              | Counter = 00001 (1) at t=65                              |
| 70        | Down=1               | —     | Down only: decrement toward 0                            | Counter = 00000 (0) at t=75; Low = 1                     |
| 110       | Load=1               | 11110 | Load Counter with 30 (11110)                             | Counter = 11110 (30) at t=115                            |
| 130       | Up=1                 | —     | Up only: increment toward 31 saturation                  | Counter = 11111 (31) at t=135; High = 1                  |

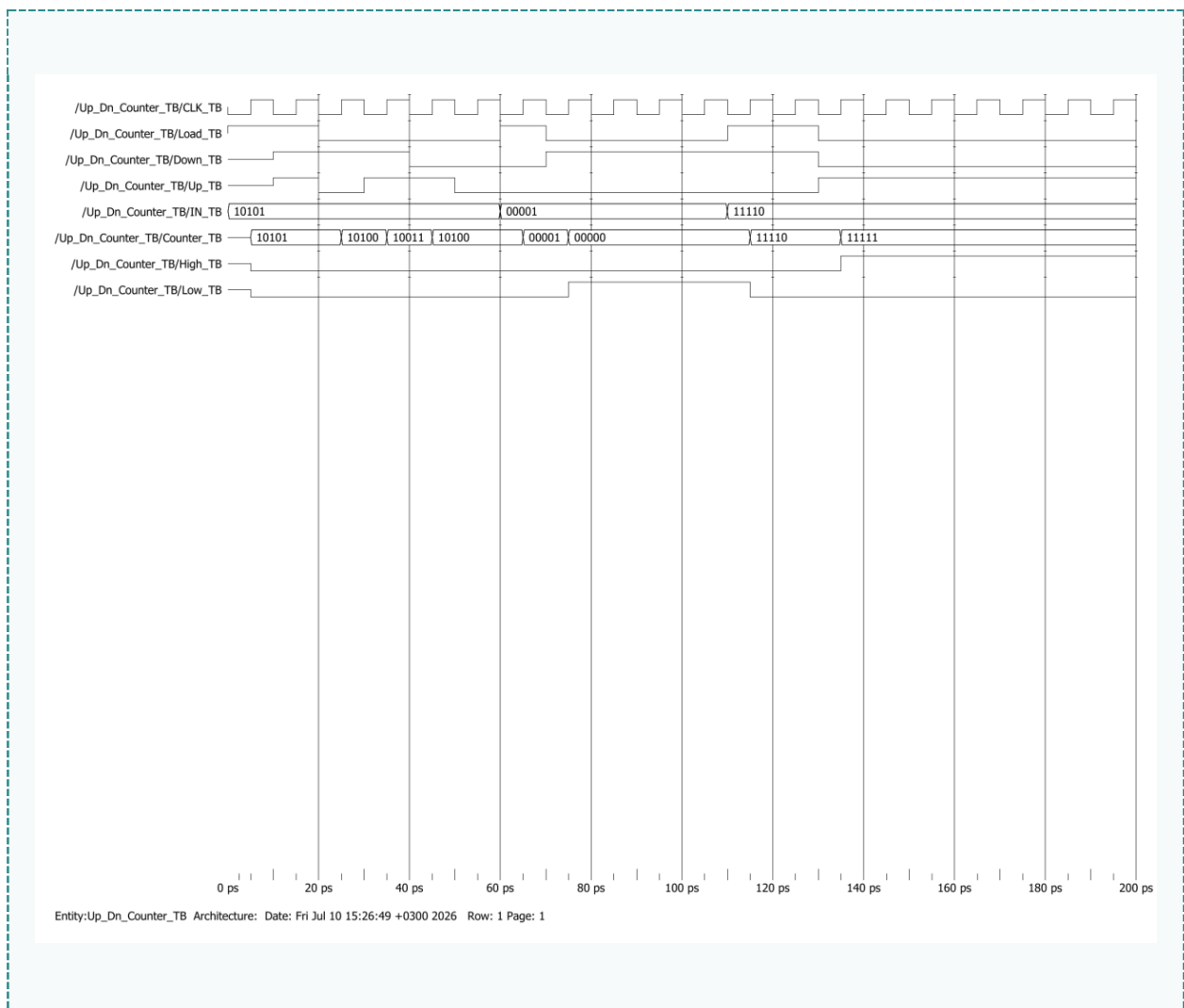


Figure 2: ModelSim simulation waveform for Up\_Dn\_Counter\_TB (CLK, Load, Down, Up, IN, registered Counter, High/Low flags)

## 05 DISCUSSION

The simulation results match the specification exactly. The transcript confirms correct loading at  $t=5$  (21),  $t=65$  (1), and  $t=115$  (30); correct decrementing to 20, 19, then back to 20 over  $t=25-45$ ; a correctly held value during the no-signal cycle at  $t=55$ ; correct saturation at 0 with the Low flag asserted at  $t=75$ ; and correct saturation at 31 with the High flag asserted at  $t=135$ . The two priority-conflict cases were the most important to verify: at  $t=10$  Load, Down, and Up are all asserted together, and the counter correctly follows Load (highest priority) rather than Down or Up; at  $t=30$  Down and Up are asserted together with Load low, and the counter correctly follows Down (next priority) rather than Up. No discrepancies were observed between expected and simulated behavior. Since this is a purely combinational/register-level design targeting behavioral simulation, no synthesis or timing analysis trade-offs (area vs. speed, critical path) arise for this assignment.

## 06 CONCLUSION

The Up\_Dn\_Counter module was successfully designed and verified in Verilog. A priority-encoded if-else if chain proved to be a clean, simple way to implement the required Load > Down > Up precedence without a separate state machine, and gating the increment/decrement paths with the High/Low flags was sufficient to implement saturation cleanly at both bounds. Icarus Verilog simulation with a targeted, priority-conflict-aware test sequence (rather than exhaustive input coverage) was enough to confirm correct behavior across all specified cases, including both saturation boundaries.